Nanyang Technological University

SC5002 Artificial Intelligence Fundamentals & Applications

Total Marks: 100 Marks

Deadline: Week 10, Sunday (23:59 SGT) via NTULearn

Submission: Report (PDF) + GitHub Link (set to Public) + Video

Assignment Overview & AI Integration Guidelines

This course assignment integrates core AI concepts—Classical Search Algorithms and Regularized Regression Modeling—tailored specifically for non-computing students. Rather than focusing on heavy mathematical proofs or complex programming from scratch, this course emphasizes conceptual logic, applied problem-solving, and utilizing Generative AI tools (e.g., ChatGPT, Gemini, Claude) as an interactive learning partner.

MANDATORY "LEARN WITH AI" REQUIREMENT

For every task, you must include a short **AI Prompting Log** in your final report. Each log must contain:

1. The exact prompt(s) you submitted to your Generative AI tool.
2. A 2–3 sentence synthesis of the AI's explanation written in your own words, reflecting on how it improved your understanding.

Part 1: Classical Search & Problem Solving (40 Marks)

Theoretical & Conceptual Focus — No Coding Required

Task 1.1: Graph Traversal & Heuristic Search (20 Marks)

Consider the state space of a search problem defined as a directed graph.

The initial state is S, and the goal state is G.

- **Graph Edges & Costs:** $S \to A\ (2)$, $S \to B\ (1)$, $S \to F\ (3)$, $A \to C\ (2)$, $A \to D\ (3)$, $B \to D\ (2)$, $B \to E\ (4)$, $C \to G\ (4)$, $D \to G\ (4)$, $F \to G\ (6)$.
- **Admissible Heuristic Values $h(n)$:** $S = 6$, $A = 4$, $B = 5$, $C = 2$, $D = 2$, $E = 8$, $F = 4$, $G = 0$.

Questions to Answer:

1. Path Search: Trace the complete path from initial state S to goal state G using Breadth-First Search (BFS) and A* Search.
2. Evaluation Table: For A* Search, construct a step-by-step expansion table showing $g(n)$ (path cost from start), $h(n)$ (heuristic cost), and $f(n) = g(n) + h(n)$ for each expanded node.

LEARN WITH AI TASK 1.1

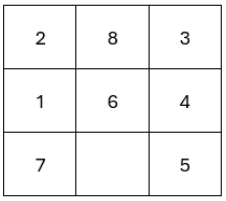
Prompting Task: Ask an AI tool to explain the concept of *"admissible heuristics"* using a real-world GPS navigation analogy.

Deliverable: Record your exact prompt, summarize the AI's explanation, and answer this question: *"If the true shortest path from node B to G costs 6, is the provided heuristic value h(B) = 5 admissible? Why or why not?"*

Task 1.2: 8-Puzzle Problem Solving (20 Marks)

Solve the 8-puzzle transition using A* Search.

Initial State: [[2, 8, 3], [1, 6, 4], [7, Blank, 5]] ➔ **Goal State:** [[1, 2, 3], [8, Blank, 4], [7, 6, 5]]



Initial state



Final state

Assume:

$g(n)$ = depth of node and $h(n)$ = number of misplaced tiles

Questions to Answer:

1. Show the steps for A* search, calculating $f(n) = g(n) + h(n)$ for each possible move to reach final state.

LEARN WITH AI TASK 1.2

After attempting the A* search manually, use an AI assistant to check your calculations and node-selection decisions

Ask your AI assistant:

"I am solving an 8-puzzle problem using A search, where g(n) is the depth of the node and h(n) is the number of misplaced tiles. I have calculated the values for each possible move. Here is my working: [insert your steps]. Check my calculations and node-selection decisions. Identify any errors or steps that need reconsideration, but do not solve the problem for me."*

Deliverable: Compare the AI's feedback with your own calculations. Identify one calculation or node-selection decision that you verified using AI. If AI identified an error, explain how you corrected it. State whether you accepted, modified, or rejected the AI's suggestion, and briefly explain why.

Part 2: Applied Machine Learning with AI Copilot (40 Marks)

Low-Code Practical Focus using Google Colab & Scikit-Learn

Task 2.1: Dataset Setup & AI Feature Analysis (10 Marks)

Open Google Colab and run Cell 1 to load the California Housing dataset. Copy the first 5 rows of output into your GenAI tool.

LEARN WITH AI TASK 2.1

Prompt Template: *"I am a non-programmer studying Machine Learning. Here are the first 5 rows from my dataset: [paste rows]. Identify: (1) The continuous target variable, (2) The input features, and (3) Why numerical feature scaling (StandardScaler) helps linear regression."*

Deliverable: Document your prompt and synthesize the AI's response in bullet points.

Task 2.2: Model Execution & Alpha Tuning (20 Marks)

Run the provided Python code block to train Linear Regression and Ridge Regression models. Observe the 5-Fold Cross-Validation R^2 scores across different regularization penalties ($\alpha = 0.1, 10.0, 1000.0$).

> **LEARN WITH AI TASK 2.2**
>
> **Prompting Task:** Paste your cross-validation results into your AI assistant.
>
> **Prompt Template:** *"My 5-fold cross-validation R² score for Linear Regression is [X], but Ridge Regression with alpha=10 scored [Y]. Explain to a non-technical manager why Ridge performed differently and why cross-validation gives a more reliable metric than a single train/test split."*
>
> **Deliverable:** Log your model evaluation scores in a table alongside your summarized AI explanation.

Task 2.3: Visual Analysis & Overfitting Insights (10 Marks)

Execute the plot cell to generate the performance comparison bar chart.

> **LEARN WITH AI TASK 2.3**
>
> **Prompt Task:** Ask AI: *"Why does setting the Ridge penalty alpha extremely high (e.g., 1000) cause model performance (R² score) to drop significantly?"*
>
> **Deliverable:** Write a 1-paragraph synthesis explaining the balance between **underfitting** and **overfitting**.

Part 3: Final Synthesis, Video & Submission (20 Marks)

Task 3.1: 5-Minute Video Presentation (10 Marks)

- Present your key findings from both Search Algorithms (Part 1) and Machine Learning Experiments (Part 2).
- **Requirements:** Include a webcam overlay showing your face. Explain how GenAI helped debug code or clarify difficult concepts. All group members must present their specific contributions.

Task 3.2: Submission Artifacts (10 Marks)

- **PDF Report:** Maximum 6 pages containing Part 1 hand calculations, AI prompt logs, and Part 2 score tables, and all deliverables of Learn with AI tasks.
- **GitHub Repository:** Submit a public repository with your completed Colab Notebook (`.ipynb`) and a detailed `README.md`.

Google Colab Code Template (Python)

Copy and paste this code block directly into a single cell in Google Colab and click Run.

```
# ================================================================================
# SC5002 - LAB ASSIGNMENT PART 2: LOW-CODE ML TEMPLATE
# Instructions: Run this cell sequentially in Google Colab.
# ================================================================================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.preprocessing import StandardScaler

# --- TASK 2.1: DATASET SETUP ---
print("=== TASK 2.1: Loading California Housing Dataset ===")
housing = fetch_california_housing(as_frame=True)
df = housing.frame.sample(n=1000, random_state=42) # Sample 1000 records

print("First 5 rows of dataset (Copy these for AI Prompt 2.1):")
print(df.head())

X = df.drop(columns=['MedHouseVal'])
y = df['MedHouseVal']

# Scale numerical features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

# --- TASK 2.2: MODEL TRAINING & CROSS-VALIDATION ---
print("=== TASK 2.2: Training Linear Regression ===")
lin_reg = LinearRegression()
lin_scores = cross_val_score(lin_reg, X_train, y_train, cv=5, scoring='r2')
print(f"Linear Regression Mean 5-Fold R2 Score: {lin_scores.mean():.4f}")

# Ridge Regression Experiments
alphas = [0.1, 10.0, 1000.0]
ridge_results = {}

print("--- Training Ridge Regression with Varying Alpha Values ---")
for a in alphas:
    ridge = Ridge(alpha=a)
    scores = cross_val_score(ridge, X_train, y_train, cv=5, scoring='r2')
    ridge_results[a] = scores.mean()
    print(f"Ridge Regression (alpha={a:6.1f})
    Mean 5-Fold R2 Score: {scores.mean():.4f}")

# --- TASK 2.3: VISUALIZATION ---
print("=== TASK 2.3: Generating Model Comparison Chart ===")
plt.figure(figsize=(9, 5))
models = ['Linear Reg'] + [f'Ridge (a={a})' for a in alphas]
scores = [lin_scores.mean()] + list(ridge_results.values())

bars = plt.bar(models, scores, color=['#3182ce', '#38a169', '#dd6b20', '#e53e3e'])
plt.ylabel('Mean 5-Fold Cross-Validation R² Score', fontsize=10)
plt.title('Linear vs. Ridge Regression Performance Comparison', fontsize=12, fontweight='bold')
plt.ylim(0, 1.0)
plt.grid(axis='y', linestyle='--', alpha=0.7)

for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2.0, yval + 0.02, f'{yval:.3f}', ha='center', va='bottom')
plt.show()
```